UT_05.1

# Comunicación asíncrona cliente-servidor (I)

Gestión de errores, JSON y programación asíncrona en JavaScript

Las aplicaciones web actuales no recargan la página entera cada vez que necesitan un dato: piden al servidor solo lo que hace falta y actualizan una parte de la interfaz. Para conseguirlo necesitamos dos piezas: un formato con el que los datos viajen de un programa a otro (**JSON**) y una forma de esperar la respuesta sin congelar la página (**programación asíncrona**). Antes de empezar repasaremos cómo se controlan los errores con `try/catch`, porque lo usaremos continuamente. En el tema siguiente (UT_05.2) lo juntaremos todo con la API **Fetch**.

## Índice

**1. Introducción ..... 2**
1.1 Qué vamos a aprender ..... 2
1.2 De recargar páginas a pedir solo datos ..... 2
**2. Gestión de errores: try/catch ..... 3**
2.1 Qué ocurre cuando hay un error ..... 3
2.2 try / catch ..... 3
2.3 Otros errores y el bloque finally ..... 4
2.4 Lanzar nuestros propios errores: throw ..... 5
2.5 ¿Cuándo usar try/catch? ..... 6
**3. JSON ..... 6**
3.1 ¿Para qué sirve JSON? ..... 6
Una palabra que oirás mucho: API REST ..... 7
3.2 Qué es JSON ..... 7
3.3 Sintaxis y tipos de datos ..... 7
3.4 JSON frente a un objeto literal de JavaScript ..... 9
3.5 Convertir objetos en texto: JSON.stringify ..... 9
Sangría para humanos ..... 10
Elegir qué propiedades se envían ..... 10
3.6 Convertir texto en objetos: JSON.parse ..... 10
3.7 Lo que se pierde por el camino ..... 11
3.8 Copiar objetos ..... 12
Valores simples y objetos ..... 12
Igualar no copia ..... 12
Copia superficial: el operador de propagación ..... 13
Copia profunda: structuredClone ..... 13
El truco de JSON ..... 14
3.9 Importar un JSON como módulo ..... 14
3.10 No te fíes de los datos externos ..... 15
**4. Programación asíncrona ..... 16**
4.1 Código síncrono y asíncrono ..... 16
4.2 setTimeout: simular la espera ..... 17
4.3 ¿En qué orden se ejecuta? ..... 18
4.4 Callbacks ..... 19
4.5 Promesas ..... 20
Reaccionar al resultado: then, catch y finally ..... 20
Crear una promesa ..... 20
4.6 async y await ..... 21

4.7 Varias promesas a la vez ..... 23
4.8 Encadenar promesas con then ..... 24
4.9 Errores habituales ..... 25
**5. Resumen, actividades y para saber más ..... 26**
5.1 Resumen ..... 26
5.2 Actividades propuestas ..... 26
5.3 Para saber más ..... 27
5.4 Siguiente paso ..... 27

> **Para recordar**
> **Material de apoyo.** Todos los listados de este tema están en la carpeta `Ejemplos/EJEM_05.1`, ya probados. Son archivos de consola que se ejecutan con Node.js 22 o superior, por ejemplo `node 01_try_catch.js`. No hace falta ningún servidor.

# 1. Introducción

## 1.1 Qué vamos a aprender

Al terminar este tema serás capaz de:

- Controlar los errores de un programa con `try/catch/finally` y lanzar errores propios con `throw`.
- Explicar qué es JSON y para qué sirve, escribirlo correctamente y convertir datos de JavaScript a JSON y viceversa.
- Copiar objetos sabiendo cuándo se comparten datos con el original (copia superficial) y cuándo no (copia profunda).
- Distinguir código síncrono y asíncrono y simular esperas con `setTimeout`.
- Escribir código asíncrono con promesas y con `async/await`.

## 1.2 De recargar páginas a pedir solo datos

Durante años cada acción del usuario (enviar un formulario, cambiar de página) hacía que el navegador pidiera al servidor un documento HTML completo y sustituyera el anterior. Hoy lo habitual es que el HTML se cargue una vez y que el código JavaScript pida **solo datos** al servidor, para pintarlos después en la página sin recargarla. Eso permite, por ejemplo:

- **Mostrar información que cambia**: listados de productos, el tiempo, mensajes nuevos.
- **Enviar datos**: crear una cuenta, publicar un comentario, guardar una configuración.
- **Buscar mientras el usuario escribe**: sugerencias y autocompletado.
- **Comprobar datos sin recargar**: ver si un nombre de usuario ya existe.

Para hacerlo bien hay que resolver dos problemas: en qué **formato** viajan esos datos (JSON, apartado 3) y cómo **esperar** la respuesta sin bloquear la página (programación asíncrona, apartado 4). Pero antes necesitamos una herramienta imprescindible: el control de errores.

# 2. Gestión de errores: try/catch

## 2.1 Qué ocurre cuando hay un error

Cuando JavaScript encuentra una instrucción que no puede ejecutar (usar una variable que no existe, leer una propiedad de `null`, interpretar un texto mal escrito...) **lanza un error**. Si nadie lo controla, el programa **se detiene en ese punto** y el resto del código no se ejecuta. En una página web eso puede significar que un botón deje de funcionar sin avisar.

Un error es un objeto que tiene, al menos, dos propiedades: `name` (el tipo de error) y `message` (la descripción). Estos son los tipos que verás con más frecuencia:

| Tipo de error | Cuándo se produce | Ejemplo |
|---|---|---|
| `ReferenceError` | Se usa una variable que no existe | `console.log(usuario)` sin haberla declarado |
| `TypeError` | Se usa un valor de forma incorrecta | Leer una propiedad de `null`: `persona.nombre` |
| `SyntaxError` | Un texto no tiene la forma correcta | `JSON.parse('{titulo: 1}')` |
| `RangeError` | Un valor está fuera del rango permitido | `new Array(-1)` |

***Tabla 1** Tipos de error más habituales.*

## 2.2 try / catch

Con `try/catch` le decimos a JavaScript: «intenta ejecutar este bloque; si algo falla, no detengas el programa, ejecuta este otro».

```
try {
  // código que puede fallar
} catch (error) {
  // qué hacer si falla; error contiene name y message
}
```

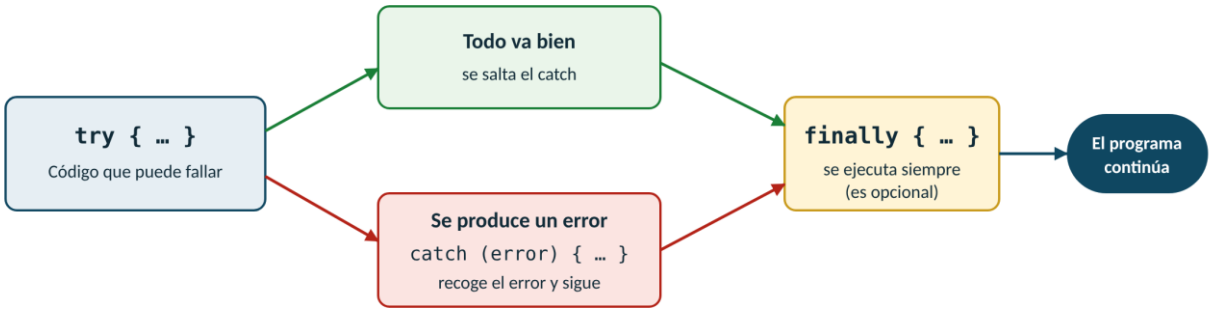



***Figura 1** Flujo de try/catch/finally: si no hay error se salta el catch; si lo hay, se ejecuta; el finally se ejecuta siempre.*

```
console.log('--- 1. try / catch: el programa continúa');
try {
  console.log('Antes del error');
  console.log(usuarioInexistente); // ReferenceError: esa variable no existe
  console.log('Esta línea no se ejecuta');
} catch (error) {
  console.log('Error capturado:', error.name, '-', error.message);
}
console.log('El programa continúa');
```

***Listado 1*** *Un error capturado: el programa no se detiene (EJEM_05.1_sencillo/01_try_catch.js).*

```
--- 1. try / catch: el programa continúa
Antes del error
Error capturado: ReferenceError - usuarioInexistente is not defined
El programa continúa
```

Fíjate en tres detalles:

- La línea que sigue al error **dentro del `try` no se ejecuta**: en cuanto algo falla, JavaScript salta al `catch`.
- El `catch` recibe el error en un parámetro (aquí `error`) con el que podemos mostrar su `name` y su `message`.
- Tras el `catch`, **el programa continúa** con normalidad.

## 2.3 Otros errores y el bloque finally

Los errores no son solo de variables que no existen. Dos muy frecuentes son leer una propiedad de `null` (`TypeError`) e interpretar como JSON un texto mal escrito (`SyntaxError`), algo que veremos a fondo en el apartado siguiente.

```
console.log('--- 2. Otros errores habituales');
const persona = null;
try {
  console.log(persona.nombre); // TypeError: no se puede leer una propiedad de null
} catch (error) {
  console.log(error.name, '-', error.message);
}
try {
  JSON.parse('{titulo: "Dune"}'); // SyntaxError: ese texto no es JSON válido
} catch (error) {
  console.log(error.name);
}
```

***Listado 2*** *Otros errores habituales (EJEM_05.1_sencillo/01_try_catch.js).*

```
--- 2. Otros errores habituales
TypeError - Cannot read properties of null (reading 'nombre')
SyntaxError
```

El bloque **`finally`** se ejecuta **siempre**, haya habido error o no. Sirve para lo que debe ocurrir en cualquier caso, por ejemplo ocultar el mensaje «Cargando...» tanto si los datos llegan como si fallan. Es opcional: puedes escribir `try/catch`, `try/finally` o los tres bloques.

```
console.log('--- 3. finally: se ejecuta siempre');
const leerEdad = (texto) => {
  try {
    const datos = JSON.parse(texto);
    return datos.edad;
  } catch (error) {
    console.log('  No se pudo leer el texto:', error.name);
    return null;
  } finally {
    console.log('  (finally) fin de la lectura');
  }
};
console.log('Edad:', leerEdad('{"edad": 19}'));
console.log('Edad:', leerEdad('{edad: 19}'));
```

***Listado 3*** *try, catch y finally (EJEM_05.1_sencillo/01_try_catch.js).*

```
--- 3. finally: se ejecuta siempre
  (finally) fin de la lectura
Edad: 19
  No se pudo leer el texto: SyntaxError
  (finally) fin de la lectura
Edad: null
```

> **Nota**
> Observa que el `finally` se ejecuta **antes** de que la función devuelva su valor: aunque haya un `return` en el `try` o en el `catch`, el `finally` siempre se cumple.

## 2.4 Lanzar nuestros propios errores: throw

No solo JavaScript lanza errores: nosotros también podemos hacerlo con `throw`. Es la forma de **rechazar datos incorrectos** y avisar a quien llamó a nuestra función de que algo va mal. Se lanza normalmente un objeto `Error` con un mensaje, o un tipo concreto (`TypeError`, `RangeError`...).

```
console.log('--- 4. Lanzar nuestros propios errores con throw');
const comprobarEdad = (edad) => {
  if (typeof edad !== 'number') throw new TypeError('La edad debe ser un número');
  if (edad < 0) throw new RangeError('La edad no puede ser negativa');
  return `Edad correcta: ${edad}`;
};
for (const valor of [25, -3, 'veinte']) {
  try {
    console.log(comprobarEdad(valor));
  } catch (error) {
    console.log(`${error.name}: ${error.message}`);
  }
}
```

***Listado 4*** *Lanzar errores con throw y capturarlos (EJEM_05.1_sencillo/01_try_catch.js).*

```
--- 4. Lanzar nuestros propios errores con throw
Edad correcta: 25
RangeError: La edad no puede ser negativa
TypeError: La edad debe ser un número
```

### 2.5 ¿Cuándo usar try/catch?

- **Úsalo con operaciones que pueden fallar por causas que no controlas**: datos que vienen de fuera (JSON, un servidor, un formulario), la conexión de red, la lectura de archivos.
- **No envuelvas todo el programa «por si acaso»**: un error de programación que queda oculto es mucho más difícil de encontrar.
- **No dejes el `catch` vacío**: como mínimo muestra el problema con `console.error(error)` o avisa al usuario.

> **Importante**
> `try/catch` solo atrapa los errores que ocurren **mientras se ejecuta el bloque**. Un error que se produce más tarde (por ejemplo, dentro de la función de un `setTimeout`) ya no está dentro del `try` y no se captura. Para el código asíncrono tendremos las herramientas adecuadas: el `.catch()` de las promesas y el `await` dentro de un `try` (apartado 4).

## 3. JSON

### 3.1 ¿Para qué sirve JSON?

Imagina que el navegador necesita la lista de libros de una tienda. Esa lista existe en el **servidor**, dentro de un programa que puede estar escrito en cualquier lenguaje (Java, PHP, Python, Node.js...) y que guarda los datos en su memoria o en una base de datos. El navegador es **otro programa**, escrito en JavaScript y situado en otro ordenador. ¿Cómo pasa la lista de uno a otro?

Por la red solo pueden viajar **bytes**, es decir, **texto**. No se puede enviar «un objeto» tal cual, porque un objeto es una estructura que vive en la memoria de un programa concreto y otro lenguaje no sabría interpretarla. Hace falta un **formato de texto acordado** por los dos lados: el emisor convierte sus datos a ese formato y el receptor los reconstruye. Ese formato, hoy omnipresente en la web, es **JSON**.

Estos son los lugares donde te lo vas a encontrar:

- En las **respuestas de los servidores** cuando se les piden datos (APIs).
- En **archivos de configuración**: `package.json`, los ajustes de Visual Studio Code...
- En los datos guardados en el navegador (`localStorage`).
- En archivos de datos y en bases de datos documentales.

#### Una palabra que oirás mucho: API REST

Una **API** es un servicio que un programa ofrece para que otros programas le pidan cosas. Una **API REST** es la forma más habitual de hacerlo en la web: cada tipo de dato tiene una **dirección** (URL) y el servidor responde con los datos en formato **JSON**. Por ejemplo, al pedir `https://api.ejemplo.com/libros/1`, el servidor podría contestar:

```json
{
  "id": 1,
  "titulo": "El nombre de la rosa",
  "autor": "Umberto Eco",
  "anio": 1980
}
```

***Listado 5*** *Respuesta imaginaria de una API REST: un libro en formato JSON.*

Pedir datos a una API desde JavaScript lo haremos con `fetch` en UT_05.2, y la unidad «Consumo de APIs REST» lo estudiará a fondo. Por ahora basta con quedarse con esta idea: **la mayor parte de los datos que usarás en el cliente llegarán en formato JSON**, de ahí la importancia de dominarlo.

## 3.2 Qué es JSON

**JSON** (*JavaScript Object Notation*) es un formato de **texto** para representar datos estructurados. Nació a partir de la sintaxis de los objetos de JavaScript, pero es independiente del lenguaje: casi todos los lenguajes pueden leerlo y escribirlo. Su tipo MIME es `application/json`.

Recuerda la idea clave: **JSON no es un objeto de JavaScript, es una cadena de texto** que *describe* datos. Por eso puede viajar por la red o guardarse en un archivo. Antes de enviarlo hay que convertir los objetos en texto (*stringify*) y, al recibirlo, convertir el texto en objetos (*parse*).

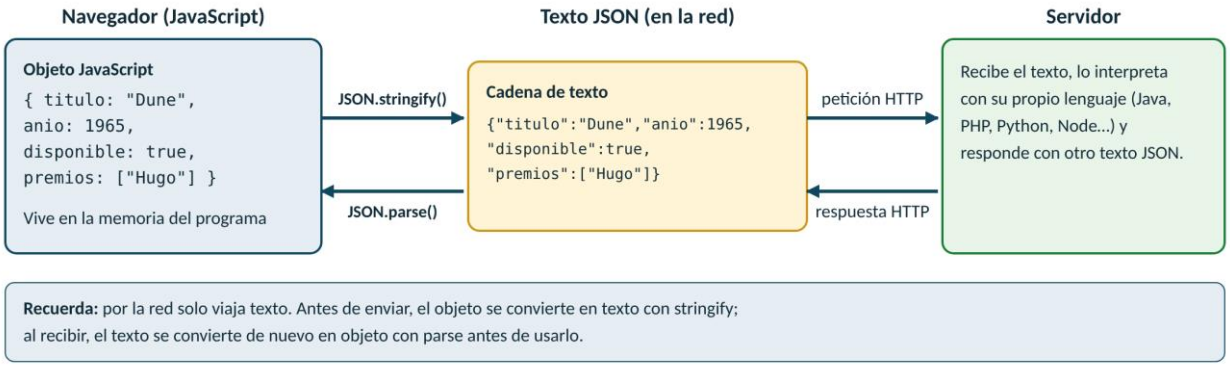



***Figura 2*** *El viaje de los datos: el emisor serializa con JSON.stringify, el texto viaja por la red y el receptor lo reconstruye con JSON.parse (y a la inversa).*

## 3.3 Sintaxis y tipos de datos

JSON admite solo seis tipos de valor. Un documento JSON es un único valor de cualquiera de ellos (lo habitual es un objeto o un array).

| Tipo JSON | Ejemplo | Observaciones |
|---|---|---|
| Objeto | `{"titulo": "Dune", "anio": 1965}` | Pares clave: valor entre llaves. Las claves son **siempre** cadenas con comillas dobles. |
| Array | `["Hugo", "Nebula"]` | Lista ordenada de valores. |
| Cadena | `"Hola"` | Solo **comillas dobles**. |
| Número | `21`, `-3.5` | Sin ceros a la izquierda, sin `NaN` ni `Infinity`. |
| Booleano | `true`, `false` | En minúsculas. |
| Nulo | `null` | Ausencia intencionada de valor. **No existe `undefined`**. |

***Tabla 2** Los seis tipos de valor de JSON.*

Las reglas que más se incumplen, y que son la causa de casi todos los `SyntaxError` que verás:

- Las claves y las cadenas llevan **comillas dobles**; las simples no valen.
- No se permite la **coma final** tras el último elemento de un objeto o array.
- No hay **comentarios**.
- No se admiten `undefined` ni funciones.

```
const textos = [
  '{"titulo":"Dune","anio":1965,"premios":["Hugo","Nebula"],"editorial":null}', // correcto
  "{'titulo':'Dune'}", // comillas simples
  '{titulo:"Dune"}', // clave sin comillas
  '{"titulo":"Dune",}', // coma final
  '{"editorial": undefined}', // undefined no existe en JSON
  'Dune', // texto sin comillas
  '"Dune"', // ¡una cadena entre comillas dobles SÍ es JSON válido!
  '42', // un número también
];

for (const texto of textos) {
  try {
    const valor = JSON.parse(texto);
    console.log('✓ válido  ', texto, '→', typeof valor);
  } catch (error) {
    console.log('✗ inválido', texto, '→', error.name);
  }
}
```

***Listado 6** Probando qué es JSON válido y qué no (EJEM_05.1_sencillo/02_json_sintaxis.js).*

```
✓ válido   {"titulo":"Dune","anio":1965,"premios":["Hugo","Nebula"],"editorial":null} →
object
✗ inválido {'titulo':'Dune'} → SyntaxError
✗ inválido {titulo:"Dune"} → SyntaxError
✗ inválido {"titulo":"Dune",} → SyntaxError
✗ inválido {"editorial": undefined} → SyntaxError
✗ inválido Dune → SyntaxError
✓ válido   "Dune" → string
✓ válido   42 → number
```

*Salida del Listado 6. El último valor del resultado indica el tipo de dato obtenido.*

> **Nota**
> Observa los dos últimos casos válidos: `"Dune"` (entre comillas dobles) y `42` son documentos JSON correctos, mientras que `Dune` sin comillas **no** lo es. Un error muy frecuente es intentar «parsear» un texto que no es JSON, por ejemplo la página HTML de error que devuelve un servidor.

## 3.4 JSON frente a un objeto literal de JavaScript

Se parecen tanto que se confunden. Compara:

```
// Objeto literal de JavaScript (código)
const libro = {
  titulo: 'Dune',          // clave sin comillas y comillas simples
  anio: 1965,
  leer() { return 1; },    // se permiten funciones
  // y también comentarios
};

// JSON (texto): una cadena que parece un objeto
const texto = '{"titulo":"Dune","anio":1965}';
```

***Listado 7** Un objeto literal es código; JSON es texto.*

| | Objeto literal de JS | JSON |
|---|---|---|
| Naturaleza | Código: crea un objeto en memoria | Texto: hay que convertirlo con `JSON.parse` |
| Claves | Con o sin comillas | Siempre entre comillas dobles |
| Cadenas | Comillas simples, dobles o plantillas | Solo comillas dobles |
| Valores | Cualquiera, incluidas funciones | Solo los seis tipos de la tabla anterior |
| Comentarios y coma final | Sí | No |

***Tabla 3** Diferencias entre un objeto literal de JavaScript y JSON.*

## 3.5 Convertir objetos en texto: JSON.stringify

`JSON.stringify(valor)` recibe un valor de JavaScript y devuelve una **cadena de texto** en formato JSON. Es lo que hacemos antes de enviar datos o de guardarlos.

```
const libro = {
  titulo: 'Dune',
  anio: 1965,
  disponible: true,
  etiquetas: ['ciencia ficción', 'saga'],
};

console.log('--- 1. De objeto a texto: JSON.stringify');
const texto = JSON.stringify(libro);
console.log(texto);
console.log(typeof texto); // string
```

***Listado 8** Convertir un objeto en texto JSON (EJEM_05.1_sencillo/03_stringify_parse.js).*

```
--- 1. De objeto a texto: JSON.stringify
{"titulo":"Dune","anio":1965,"disponible":true,"etiquetas":["ciencia ficción","saga"]}
string
```

### Sangría para humanos

Por defecto el texto sale compacto, que es lo ideal para enviarlo por la red. Si pasas `null` como segundo argumento y un número como tercero, añade saltos de línea y esa cantidad de espacios de sangría, algo útil para depurar o para archivos legibles.

```
console.log('--- 2. Con sangría para leerlo mejor');
console.log(JSON.stringify(libro, null, 2));
```

***Listado 9*** *Texto con sangría.*

```
--- 2. Con sangría para leerlo mejor
{
  "titulo": "Dune",
  "anio": 1965,
  "disponible": true,
  "etiquetas": [
    "ciencia ficción",
    "saga"
  ]
}
```

### Elegir qué propiedades se envían

El segundo argumento también puede ser un **array con los nombres** de las propiedades que quieres conservar. Es útil para no enviar datos internos o privados (contraseñas, por ejemplo).

```
console.log('--- 3. Quedarnos solo con algunas propiedades');
console.log(JSON.stringify(libro, ['titulo', 'anio']));
```

***Listado 10*** *Quedarse solo con algunas propiedades.*

```
--- 3. Quedarnos solo con algunas propiedades
{"titulo":"Dune","anio":1965}
```

## 3.6 Convertir texto en objetos: JSON.parse

`JSON.parse(texto)` hace el camino inverso: recibe un texto JSON y devuelve el valor de JavaScript que describe. Si el texto **no es JSON válido, lanza un `SyntaxError`**, así que siempre que el texto venga de fuera (servidor, `localStorage`, un archivo, un formulario) debe ir dentro de un `try/catch`, como vimos en el apartado 2.

```
console.log('--- 4. De texto a objeto: JSON.parse');
const recuperado = JSON.parse(texto);
console.log(recuperado.titulo, '-', recuperado.etiquetas[0]);
console.log(typeof recuperado); // object
```

***Listado 11*** *Convertir texto JSON en un objeto (EJEM_05.1_sencillo/03_stringify_parse.js).*

```
--- 4. De texto a objeto: JSON.parse
Dune - ciencia ficción
object
```

> **Atención**
> Para leer JSON usa siempre `JSON.parse`. **Nunca** uses `eval()` para «convertir» un texto en un objeto: ejecutaría cualquier código que contuviera el texto.

## 3.7 Lo que se pierde por el camino

Como JSON solo conoce seis tipos de valor, al convertir a texto algunas cosas de JavaScript **se transforman o desaparecen**:

| Valor de JavaScript | Qué ocurre con `JSON.stringify` |
|---|---|
| `undefined` y funciones | La propiedad **desaparece** |
| `NaN` e `Infinity` | Se convierten en `null` |
| Fechas (`Date`) | Se convierten en **texto**, y con `JSON.parse` siguen siendo texto |

***Tabla 4*** *Valores que no sobreviven al viaje.*

```
console.log('--- 5. Lo que se pierde por el camino');
const datos = {
  nombre: 'Ana',
  apodo: undefined, // se omite
  saludar() { return 'hola'; }, // se omite
  nota: NaN, // pasa a null
  registro: new Date('2026-01-15T10:00:00Z'), // pasa a texto
};
const ida = JSON.stringify(datos);
console.log(ida);
const vuelta = JSON.parse(ida);
console.log('La fecha vuelve como:', typeof vuelta.registro); // string
```

***Listado 12*** *Qué se pierde al convertir a JSON y volver (EJEM_05.1_sencillo/03_stringify_parse.js).*

```
--- 5. Lo que se pierde por el camino
{"nombre":"Ana","nota":null,"registro":"2026-01-15T10:00:00.000Z"}
La fecha vuelve como: string
```

> **Nota**
> Si necesitas recuperar una fecha, hay que reconstruirla a mano a partir del texto, por ejemplo con `new Date(texto)`. Ocurre lo mismo con cualquier dato que no sea de los seis tipos de JSON.

## 3.8 Copiar objetos

Copiar un objeto parece trivial, pero es una de las fuentes de errores más frecuentes en JavaScript. Para entenderlo hay que saber **qué guarda realmente una variable**. La Figura siguiente resume las tres situaciones que vamos a estudiar.

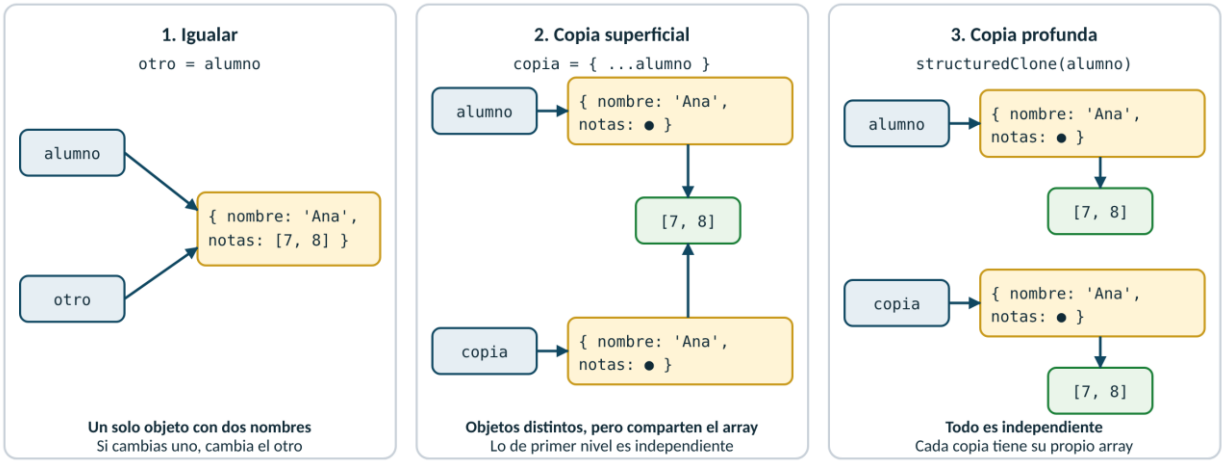



***Figura 3*** *Igualar, copia superficial y copia profunda de un objeto que contiene un array.*

## Valores simples y objetos

Una variable que contiene un número, un texto o un booleano guarda **el valor**. Una variable que contiene un objeto o un array guarda una **referencia**: algo parecido a la dirección donde está ese objeto en la memoria. Al copiar una variable se copia lo que guarda; por eso, con objetos se copia **la dirección, no el objeto**.

```
console.log('--- 1. Con números y textos, copiar copia el valor');
let a = 5;
let b = a;
b = 10;
console.log(a, b); // 5 10: son independientes
```

***Listado 13*** *Copiar un número copia el valor (EJEM_05.1_sencillo/04_copiar_objetos.js).*

```
--- 1. Con números y textos, copiar copia el valor
5 10
```

## Igualar no copia

Si escribes `otro = alumno`, no se crea ningún objeto nuevo: las dos variables **apuntan al mismo objeto**. Cualquier cambio hecho desde una se ve desde la otra.

```
console.log('--- 2. Con objetos, igualar NO copia: es el mismo objeto');
const alumno = { nombre: 'Ana', notas: [7, 8] };
const otro = alumno;
otro.nombre = 'Luis';
console.log(alumno.nombre); // Luis: ¡también ha cambiado!
console.log(alumno === otro); // true: es el mismo objeto
```

***Listado 14*** *Dos nombres para el mismo objeto.*

```
--- 2. Con objetos, igualar NO copia: es el mismo objeto
Luis
true
```

## Copia superficial: el operador de propagación

Para obtener un objeto nuevo se puede usar el operador de propagación `{ ...objeto }` (para arrays, `[...array]`). Crea un objeto nuevo y copia **sus propiedades de primer nivel**. Pero si una propiedad contiene a su vez un objeto o un array, lo que se copia es la **referencia**: el objeto interior sigue **compartido**. Por eso se llama copia **superficial**.

```
console.log('--- 3. Copia superficial con el operador de propagación (...)');
const ana = { nombre: 'Ana', notas: [7, 8] };
const copia = { ...ana };
copia.nombre = 'Luis';
console.log(ana.nombre, copia.nombre); // Ana Luis: independientes
copia.notas.push(10);
console.log(ana.notas); // [7, 8, 10]: ¡el array interior SÍ se comparte!
console.log(ana.notas === copia.notas); // true
```

***Listado 15*** *Una copia superficial comparte los objetos interiores.*

```
--- 3. Copia superficial con el operador de propagación (...)
Ana Luis
[ 7, 8, 10 ]
true
```

> **Atención**
> Es el fallo más típico: «copio» un objeto, modifico la copia y, sin querer, cambia también el original. En una interfaz, eso acaba en pantallas que muestran datos incoherentes.

## Copia profunda: structuredClone

`structuredClone(objeto)` copia el objeto **y todo lo que contiene**, a cualquier profundidad. Está disponible en todos los navegadores actuales y en Node.js desde la versión 17.

```
console.log('--- 4. Copia profunda con structuredClone');
const original = { nombre: 'Ana', notas: [7, 8] };
const profunda = structuredClone(original);
profunda.notas.push(10);
console.log(original.notas, profunda.notas); // [7, 8] [7, 8, 10]: independientes
console.log(original.notas === profunda.notas); // false
```

***Listado 16*** *Una copia profunda es totalmente independiente.*

```
--- 4. Copia profunda con structuredClone
[ 7, 8 ] [ 7, 8, 10 ]
false
```

Tiene un límite: **no sabe copiar funciones**. Si el objeto contiene alguna, lanza un error de tipo `DataCloneError`.

```
console.log('--- 5. Con funciones dentro, structuredClone falla');
try {
  structuredClone({ nombre: 'Ana', saludar() {} });
} catch (error) {
  console.log(error.name); // DataCloneError
}
```

***Listado 17*** *structuredClone no copia funciones.*

```
--- 5. Con funciones dentro, structuredClone falla
DataCloneError
```

### El truco de JSON

En tutoriales y en código antiguo verás `JSON.parse(JSON.stringify(objeto))` como forma de hacer una copia profunda. Funciona con datos sencillos, pero arrastra las pérdidas que acabamos de ver (funciones, `undefined`, fechas convertidas en texto...). Hoy se recomienda `structuredClone`.

| Forma de copiar | Qué copia | Cuándo usarla |
|---|---|---|
| `otro = objeto` | Nada: es el mismo objeto | Cuando quieres dos nombres para un mismo objeto |
| `{ ...objeto }` · `[...array]` | Solo el primer nivel (superficial) | Objetos sin objetos ni arrays dentro |
| `structuredClone(objeto)` | Todo (profunda) | Cuando necesitas una copia totalmente independiente |
| `JSON.parse(JSON.stringify(objeto))` | Todo, pero solo datos JSON | Solo en código antiguo; mejor evitarlo |

***Tabla 5*** *Formas de copiar un objeto.*

## 3.9 Importar un JSON como módulo

Cuando los datos viajan **empaquetados con la aplicación** (los textos de la interfaz, una lista de países...) no hace falta pedirlos al servidor: un módulo ES puede importar directamente un archivo JSON. Hay que indicar el tipo con un **atributo de importación**:

```js
import libros from './datos/libros.json' with { type: 'json' };

console.log(`Hay ${libros.length} libros en el archivo:`);
for (const libro of libros) {
  console.log(`- ${libro.titulo} (${libro.anio})`);
}
```

***Listado 18*** *Importar un archivo JSON como módulo (EJEM_05.1_sencillo/06_json_modulo.js).*

```
Hay 3 libros en el archivo:
- El nombre de la rosa (1980)
- Cien años de soledad (1967)
- Fahrenheit 451 (1953)
```

> **Nota**
> Los atributos de importación (`with { type: 'json' }`) funcionan en los navegadores actuales y en Node 22. Antes se escribía `assert { type: 'json' }`, que está **obsoleto**. Si los datos están en un servidor y pueden cambiar, no se importan: se piden con `fetch` (UT_05.2).

## 3.10 No te fíes de los datos externos

Que un texto sea JSON **válido** no garantiza que tenga la **forma** que tu programa espera: pueden faltar propiedades, el año puede llegar como texto o el valor puede ser `null`. Hay dos defensas:

- `try/catch` alrededor de `JSON.parse`, para el texto roto.
- **Validar la forma** de los datos antes de usarlos.

```
// 1. Lectura segura: JSON.parse lanza un error si el texto está mal escrito
const leerJSON = (texto) => {
  try {
    return JSON.parse(texto);
  } catch (error) {
    console.log('   (el texto no es JSON válido)');
    return null;
  }
};

// 2. Validación: que sea JSON válido no garantiza que tenga la forma que esperamos
const esLibroValido = (libro) => {
  if (typeof libro !== 'object' || libro === null) return false;
  if (typeof libro.titulo !== 'string' || libro.titulo === '') return false;
  if (typeof libro.anio !== 'number') return false;
  return true;
};

const entradas = [
  '{"titulo":"Dune","anio":1965}', // correcto
  '{"titulo":"Dune","anio":"1965"}', // JSON válido, pero el año es un texto
  '{"titulo":"","anio":1965}', // JSON válido, pero el título está vacío
  '{"titulo":"Dune","anio":1965', // JSON roto (falta la llave final)
  'null', // JSON válido, pero no es un objeto
];

for (const texto of entradas) {
  const libro = leerJSON(texto);
  console.log(esLibroValido(libro) ? '✓ aceptado ' : '✗ rechazado', texto);
}
```

***Listado 19*** *Lectura segura y validación de datos (EJEM_05.1_sencillo/05_validar_json.js).*

```
✓ aceptado  {"titulo":"Dune","anio":1965}
✗ rechazado {"titulo":"Dune","anio":"1965"}
✗ rechazado {"titulo":"","anio":1965}
   (el texto no es JSON válido)
✗ rechazado {"titulo":"Dune","anio":1965
✗ rechazado null
```

> **Para recordar**
> En proyectos reales la validación se suele delegar en bibliotecas como **Zod** o **Ajv**, que describen una sola vez la forma de los datos. Para empezar, basta con la validación manual.

# 4. Programación asíncrona

## 4.1 Código síncrono y asíncrono

En el código **síncrono** cada instrucción espera a que termine la anterior. Es cómodo de razonar, pero tiene un problema: si una instrucción tarda (pedir un dato a un servidor puede llevar desde 50 ms hasta

varios segundos), todo lo demás se queda parado. En una página web eso significa que no se puede pulsar un botón ni desplazar el contenido.

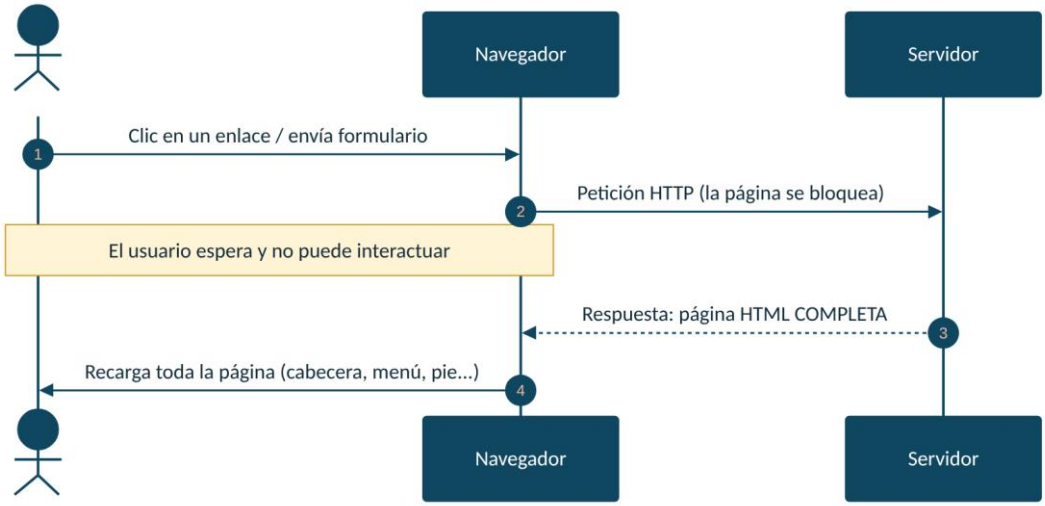



***Figura 4*** *Ejecución síncrona: mientras se espera la respuesta del servidor, la página queda bloqueada.*

En el código **asíncrono** se lanza la operación lenta y se sigue trabajando; cuando la operación termina, se avisa y se ejecuta el código que esperaba el resultado.

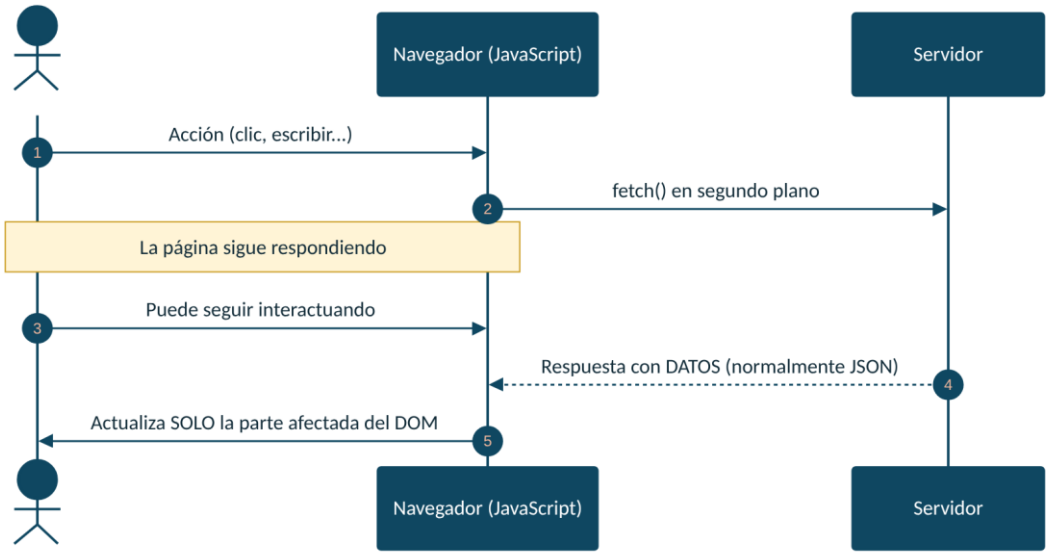



***Figura 5*** *Ejecución asíncrona: la petición se hace «en segundo plano» y la página sigue respondiendo.*

JavaScript ejecuta tu código en **un único hilo**: solo puede hacer **una cosa a la vez**. Entonces, ¿cómo puede una página seguir funcionando mientras espera 2 segundos a un servidor? Porque **la espera no la hace JavaScript**: la hace el navegador. JavaScript le encarga el trabajo («avísame dentro de 2 segundos», «avísame cuando lleguen los datos») y sigue con lo siguiente. Cuando el navegador tiene el resultado, avisa, y JavaScript ejecuta el código que estaba esperando.

## 4.2 setTimeout: simular la espera

`setTimeout(función, milisegundos)` le pide al navegador que ejecute una función **cuando pasen** los milisegundos indicados (1000 ms = 1 segundo). Es la forma más sencilla de ver la asincronía en acción, y la usaremos mucho para **simular** operaciones lentas, como una petición a un servidor, sin necesidad de

tener uno. Así podremos practicar promesas y `async/await` con ejemplos que se ejecutan en cualquier ordenador.

```
console.log('A: empiezo');

setTimeout(() => {
  console.log('B: han pasado 1000 ms');
}, 1000);

console.log('C: sigo trabajando');

// Con 0 ms también va después: primero termina el código que se está ejecutando
setTimeout(() => {
  console.log('D: setTimeout con 0 ms');
}, 0);

// Simular que pedimos datos a un servidor lento (tarda 2 segundos)
console.log('Pidiendo datos al "servidor"...');
setTimeout(() => {
  const usuario = { nombre: 'Ana', edad: 19 };
  console.log('Datos recibidos:', usuario);
}, 2000);

console.log('Mientras tanto, el programa sigue funcionando');

// Error muy común: PASAR la función frente a LLAMARLA
// const saludar = () => console.log('¡hola!');
// setTimeout(saludar(), 100);  // MAL: se ejecuta ya y se pasa undefined
// setTimeout(saludar, 100);    // BIEN: se pasa la función, sin paréntesis
```

***Listado 20** setTimeout y el orden de ejecución (EJEM_05.1_sencillo/07_settimeout.js).*

```
A: empiezo
C: sigo trabajando
Pidiendo datos al "servidor"...
Mientras tanto, el programa sigue funcionando
D: setTimeout con 0 ms
B: han pasado 1000 ms
Datos recibidos: { nombre: 'Ana', edad: 19 }
```

Observa lo que ocurre:

- **«C» sale antes que «B»**: JavaScript no se queda esperando al temporizador, continúa con la línea siguiente.
- **Un temporizador de 0 ms también sale después**: «0 ms» no significa «ya», sino «en cuanto termine el código que se está ejecutando».
- El «servidor» tarda dos segundos, pero el programa no se bloquea: el mensaje «Mientras tanto...» sale **enseguida**.

> **Importante**
> Error muy común: `setTimeout(saludar(), 100)` **llama** a `saludar` inmediatamente y pasa su resultado (`undefined`) al temporizador. Hay que **pasar** la función, sin paréntesis: `setTimeout(saludar, 100)`.

## 4.3 ¿En qué orden se ejecuta?

Con lo que sabemos ya podemos explicar la salida del ejemplo anterior. Sigue el programa paso a paso:

| Momento | Qué ocurre | Se muestra |
|---|---|---|
| 0 ms | Se ejecuta el primer `console.log` | A: empiezo |
| 0 ms | Se encarga al navegador un temporizador de 1000 ms (el de B) y se sigue adelante | — |
| 0 ms | Se ejecuta el siguiente `console.log` | C: sigo trabajando |
| 0 ms | Se encargan otro temporizador de 0 ms (D) y uno de 2000 ms (el «servidor»), y se muestran sus mensajes inmediatos | Pidiendo datos... · Mientras tanto... |
| Al acabar el código | JavaScript queda libre y ejecuta lo que ya estaba listo: el temporizador de 0 ms | D: setTimeout con 0 ms |
| 1000 ms | Vence el temporizador de B | B: han pasado 1000 ms |
| 2000 ms | Vence el temporizador del «servidor» | Datos recibidos: ... |

***Tabla 6*** *Orden de ejecución del Listado anterior.*

> **Para recordar**
> **Regla de oro:** primero se ejecuta **todo el código que está escrito de seguido**; las funciones que esperan algo (temporizadores, respuestas de red, clics) se ejecutan **después**, cuando les toca y JavaScript está libre. Este mecanismo se llama **bucle de eventos** (*event loop*) y tiene más detalles, que no hace falta conocer para empezar.

## 4.4 Callbacks

Ya has usado un *callback* sin saberlo: la función que le pasamos a `setTimeout`. Un **callback** es una **función que se pasa como argumento a otra función para que esta la ejecute más tarde**, cuando termine su trabajo. Es el mecanismo original de la asincronía en JavaScript: los eventos (`addEventListener`), los temporizadores y las primeras bibliotecas para pedir datos al servidor funcionaban así.

Funcionan bien para cosas sencillas, pero cuando hay que encadenar operaciones que dependen unas de otras aparecen tres problemas, que explican por qué se inventaron las promesas:

- **No se puede devolver el resultado con `return`.** Cuando el resultado llega, la función que lo pidió ya ha terminado, así que el resultado solo existe dentro del callback.
- **`try/catch` no sirve.** El error ocurre más tarde, cuando el `try` ya terminó, y cada callback tiene que comprobar el error por su cuenta.
- **El código se anida.** Si hay que hacer tres cosas una tras otra, cada una va dentro del callback de la anterior y el código forma una «pirámide» que se desplaza cada vez más a la derecha y es difícil de leer. Se le llama *callback hell* o «pirámide de la perdición».

> **Nota**
> No vas a escribir `callbacks` anidados: las promesas y `async/await` (apartados siguientes) resuelven estos problemas. Lo importante es **reconocer el patrón** cuando lo veas en código antiguo o en bibliotecas.

## 4.5 Promesas

Una **promesa** (`Promise`) es un objeto que representa un resultado que **todavía no existe, pero llegará** (o fallará). Piensa en el resguardo que te dan al encargar un producto en una tienda: aún no tienes el producto, pero tienes algo que representa tu pedido y con lo que decides qué hacer cuando llegue (o si no llega).

Una promesa está siempre en uno de tres estados: **pendiente** (aún no hay resultado), **cumplida** (terminó bien y tiene un valor) o **rechazada** (falló y tiene un motivo). Cuando sale de «pendiente» ya no vuelve a cambiar.

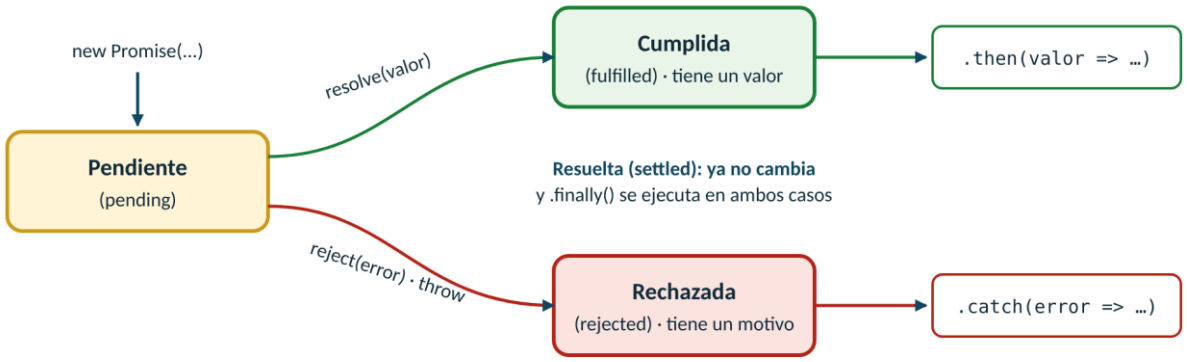



***Figura 6*** *Estados de una promesa y métodos para reaccionar a ellos.*

### Reaccionar al resultado: then, catch y finally

| Método | Se ejecuta cuando... |
|---|---|
| `.then(función)` | La promesa se **cumple**; la función recibe el valor |
| `.catch(función)` | La promesa se **rechaza**; la función recibe el error |
| `.finally(función)` | La promesa termina, **de una forma u otra** |

***Tabla 7*** *Métodos para reaccionar a una promesa.*

### Crear una promesa

Se crean con `new Promise((resolver, rechazar) => { ... })`. Dentro se hace el trabajo y se llama a `resolver(valor)` si todo va bien o a `rechazar(error)` si falla. En el trabajo diario se **consumen** promesas que ya devuelven las funciones del navegador (como `fetch`); crearlas a mano sirve sobre todo para simular, como haremos aquí con `setTimeout`.

```js
const todoBien = true;

const promesa = new Promise((resolver, rechazar) => {
  setTimeout(() => {
    if (todoBien) resolver('¡Datos recibidos!');
    else rechazar(new Error('Ha fallado la conexión'));
  }, 1000);
});

console.log('Estado al crearla:', promesa); // Promise { <pending> }

promesa
  .then((mensaje) => console.log('✓ then:', mensaje))
  .catch((error) => console.log('✗ catch:', error.message))
  .finally(() => console.log('finally: se ejecuta siempre'));

console.log('Esto sale antes de que la promesa termine');
```

***Listado 21*** *Crear una promesa y reaccionar con then, catch y finally (EJEM_05.1_sencillo/08_primera_promesa.js).*

```
Estado al crearla: Promise { <pending> }
Esto sale antes de que la promesa termine
✓ then: ¡Datos recibidos!
finally: se ejecuta siempre
```

*Salida con todoBien = true.*

```
Estado al crearla: Promise { <pending> }
Esto sale antes de que la promesa termine
✗ catch: Ha fallado la conexión
finally: se ejecuta siempre
```

*Salida con todoBien = false: se ejecuta el catch en lugar del then.*

Fíjate en que al mostrar la promesa recién creada aparece `Promise { <pending> }`, y en que la última línea del programa sale **antes** que el resultado: la promesa sigue pendiente mientras el programa continúa.

## 4.6 async y await

Las promesas resuelven los problemas de los callbacks, pero escribir `.then` y `.catch` sigue siendo menos natural que el código síncrono. **`async/await`** nos permite **escribir código asíncrono con aspecto de código síncrono**, de arriba abajo, usando las mismas promesas por debajo. Hoy es la forma recomendada de trabajar con ellas.

En los ejemplos que siguen usaremos `buscarLibro(id)`, una función que **simula pedir un libro a un servidor**: devuelve una promesa que se cumple con el libro a los 0,5 segundos, o que se rechaza si el `id` no existe. En UT_05.2 será `fetch` quien nos devuelva promesas de verdad.

- Una función marcada con **`async`** devuelve **siempre una promesa**.
- **`await`** delante de una promesa **pausa esa función** (no el programa entero) hasta que la promesa se cumple, y devuelve su valor. Si la promesa se rechaza, `await` **lanza un error**.
- Los errores se gestionan con el **`try/catch/finally`** que ya conoces del apartado 2.

- `await` solo se puede usar dentro de una función `async` o en el nivel superior de un **módulo** (*top-level await*), como hacemos en los ejemplos.

```js
// "Servidor" simulado: devuelve una promesa que tarda 0,5 s
const titulos = {
  1: 'El nombre de la rosa',
  2: 'Cien años de soledad',
  3: 'Fahrenheit 451',
};
const buscarLibro = (id) =>
  new Promise((resolver, rechazar) => {
    setTimeout(() => {
      if (titulos[id]) resolver({ id, titulo: titulos[id] });
      else rechazar(new Error(`No existe ningún libro con id ${id}`));
    }, 500);
  });

// Una función async: dentro podemos usar await
const mostrarLibro = async (id) => {
  try {
    console.log(`Buscando el libro ${id}...`);
    const libro = await buscarLibro(id); // espera a que la promesa se cumpla
    console.log('✓ Encontrado:', libro.titulo);
  } catch (error) {
    console.log('✗ Error:', error.message);
  } finally {
    console.log('Búsqueda terminada');
  }
};

console.log('--- 1. await pausa la función, no el programa');
const busqueda = mostrarLibro(1);
console.log('Esto se ejecuta mientras se busca el libro');
await busqueda; // esperamos aquí para que la salida salga en orden

console.log('--- 2. Si la promesa se rechaza, el catch recoge el error');
await mostrarLibro(99);

console.log('--- 3. Peticiones que dependen una de otra');
const primero = await buscarLibro(1);
const segundo = await buscarLibro(primero.id + 1); // necesita el resultado anterior
console.log(`${primero.titulo} → ${segundo.titulo}`);
```

***Listado 22*** *async/await con try/catch/finally (EJEM_05.1_sencillo/09_async_await.js).*

```
--- 1. await pausa la función, no el programa
Buscando el libro 1...
Esto se ejecuta mientras se busca el libro
✓ Encontrado: El nombre de la rosa
Búsqueda terminada
--- 2. Si la promesa se rechaza, el catch recoge el error
Buscando el libro 99...
✗ Error: No existe ningún libro con id 99
Búsqueda terminada
--- 3. Peticiones que dependen una de otra
El nombre de la rosa → Cien años de soledad
```

Cada parte del ejemplo muestra una idea:

- **Parte 1.** La línea «Esto se ejecuta mientras se busca el libro» sale antes que el resultado: `await` pausa la función `mostrarLibro`, pero el resto del programa sigue.
- **Parte 2.** Al buscar un libro que no existe, la promesa se rechaza, `await` lanza el error y lo recoge el `catch`. El `finally` se ejecuta igualmente.
- **Parte 3.** Cuando una petición **necesita el resultado de la anterior**, se escriben dos `await` seguidos y el código se lee como una receta, de arriba abajo.

> **Importante**
> Si te olvidas del `await`, no obtienes el valor sino una **promesa pendiente**: `const libro = buscarLibro(1)` guarda una `Promise`, no un libro, y `libro.titulo` valdrá `undefined`.

## 4.7 Varias promesas a la vez

Si las operaciones son **independientes** (por ejemplo, pedir tres libros distintos), no tiene sentido esperar una detrás de otra: se lanzan todas a la vez y se espera al conjunto. Así el tiempo total es el de la más lenta, y no la suma de todas.

| Método | Se cumple cuando... | Se rechaza cuando... |
|---|---|---|
| `Promise.all([...])` | Se cumplen **todas**. Devuelve un array con los resultados, en el mismo orden | Falla **una** cualquiera (rechaza al instante) |
| `Promise.allSettled([...])` | Terminan **todas**, bien o mal. Devuelve el estado de cada una | Nunca |

***Tabla 8*** *Los dos métodos más usados para combinar promesas.*

```
const segundosDesde = (inicio) => ((Date.now() - inicio) / 1000).toFixed(1);

console.log('--- 1. Una detrás de otra: el tiempo se suma');
let inicio = Date.now();
await buscarLibro(1);
await buscarLibro(2);
await buscarLibro(3);
console.log(`Tres libros en ${segundosDesde(inicio)} s`);

console.log('--- 2. Todas a la vez (Promise.all): el tiempo de la más lenta');
inicio = Date.now();
const libros = await Promise.all([buscarLibro(1), buscarLibro(2), buscarLibro(3)]);
console.log(`Tres libros en ${segundosDesde(inicio)} s`);
console.log(libros.map((libro) => libro.titulo));

console.log('--- 3. Si una falla, Promise.all falla');
try {
  await Promise.all([buscarLibro(1), buscarLibro(99)]);
} catch (error) {
  console.log('✗ Promise.all:', error.message);
}

console.log('--- 4. Promise.allSettled: el resultado de cada una');
const resultados = await Promise.allSettled([buscarLibro(1), buscarLibro(99)]);
for (const resultado of resultados) {
  if (resultado.status === 'fulfilled') {
    console.log('✓ cumplida :', resultado.value.titulo);
  } else {
    console.log('✗ rechazada:', resultado.reason.message);
  }
}
```

***Listado 23*** *Una detrás de otra frente a todas a la vez (EJEM_05.1_sencillo/10_varias_promesas.js; usa la misma función buscarLibro).*

```
--- 1. Una detrás de otra: el tiempo se suma
Tres libros en 1.5 s
--- 2. Todas a la vez (Promise.all): el tiempo de la más lenta
Tres libros en 0.5 s
[ 'El nombre de la rosa', 'Cien años de soledad', 'Fahrenheit 451' ]
--- 3. Si una falla, Promise.all falla
✗ Promise.all: No existe ningún libro con id 99
--- 4. Promise.allSettled: el resultado de cada una
✓ cumplida : El nombre de la rosa
✗ rechazada: No existe ningún libro con id 99
```

*Salida (los tiempos pueden variar una décima).*

> **Para recordar**
> Regla práctica: si una petición **necesita el resultado de la anterior**, usa un `await` detrás de otro; si son **independientes**, lánzalas juntas con `await Promise.all([...])`. Existen también `Promise.race` y `Promise.any`, que se verán cuando hagan falta.

## 4.8 Encadenar promesas con then

Antes de que existiera `async/await`, las promesas se usaban encadenando `then`. Es el estilo que seguirás encontrando en muchos tutoriales y proyectos, así que conviene saber leerlo. Cada `then` devuelve **otra promesa**, de modo que se pueden **encadenar**: lo que devuelve un `then` lo recibe el

siguiente, y si devuelve una promesa, la cadena espera a que se cumpla. Un error salta todos los `then` hasta el primer `catch`.

```
console.log('--- 1. Cada then devuelve una promesa');
await buscarLibro(1)
  .then((libro) => {
    console.log('1.º:', libro.titulo);
    return buscarLibro(2); // devolvemos la promesa: la cadena espera
  })
  .then((libro) => {
    console.log('2.º:', libro.titulo);
    return buscarLibro(99); // este libro no existe: la promesa se rechaza
  })
  .then((libro) => console.log('3.º:', libro.titulo)) // no se ejecuta
  .catch((error) => console.log('✗ Error:', error.message))
  .finally(() => console.log('Cadena terminada'));

console.log('--- 2. Error típico: olvidar el return');
await buscarLibro(1)
  .then((libro) => {
    buscarLibro(2); // ¡sin return! la cadena no espera
  })
  .then((resultado) => console.log('El siguiente then recibe:', resultado));
```

***Listado 24*** *Encadenar promesas con then (EJEM_05.1_sencillo/11_encadenar_then.js; usa la misma función buscarLibro).*

```
--- 1. Cada then devuelve una promesa
1.º: El nombre de la rosa
2.º: Cien años de soledad
✗ Error: No existe ningún libro con id 99
Cadena terminada
--- 2. Error típico: olvidar el return
El siguiente then recibe: undefined
```

> **Atención**
> El fallo más típico al encadenar es **olvidar el `return`** dentro de un `then`: la cadena no espera a la promesa interior y el siguiente `then` recibe `undefined`.

Compara esta cadena con el apartado 4.6: el mismo flujo escrito con `await` necesita menos símbolos y se lee de arriba abajo. Por eso, al escribir código nuevo, usaremos `async/await`.

## 4.9 Errores habituales

| Error | Síntoma | Solución |
|---|---|---|
| `setTimeout(f(), ms)` | La función se ejecuta al momento | `setTimeout(f, ms)` |
| Devolver con `return` el resultado de un callback | La función devuelve `undefined` | Usar una promesa o `async/await` |
| Olvidar el `await` | Se obtiene una promesa pendiente, no el valor | Escribir `await` delante de la llamada |
| Olvidar el `return` en un `then` | El siguiente `then` recibe `undefined` | Devolver siempre la promesa |
| `await` en serie de tareas independientes | Se tarda la suma de los tiempos | `Promise.all` |

| Error | Síntoma | Solución |
|---|---|---|
| Olvidar el `catch` o el `try/catch` | Error sin controlar en la consola | Gestionar siempre el posible fallo |

***Tabla 9*** *Errores habituales con código asíncrono.*

# 5. Resumen, actividades y para saber más

## 5.1 Resumen

- `try/catch/finally` controla los errores sin detener el programa; `throw` lanza errores propios. Se usa con operaciones que pueden fallar por causas ajenas.
- **JSON** es un formato de **texto** con el que se intercambian datos entre programas; las **API REST** responden con JSON.
- JSON tiene seis tipos de valor y reglas estrictas: comillas dobles, sin comentarios, sin coma final y sin `undefined`.
- `JSON.stringify` convierte un objeto en texto y `JSON.parse` el texto en objeto (dentro de un `try/catch`). Las funciones, `undefined` y las fechas **no sobreviven** tal cual.
- Con objetos, **igualar no copia**; `{ ...objeto }` hace una copia **superficial** y `structuredClone` una copia **profunda**.
- JavaScript tiene un solo hilo: `setTimeout` ejecuta una función más tarde y el código escrito de seguido se ejecuta **antes** que lo que espera.
- Una **promesa** puede estar pendiente, cumplida o rechazada. **`async/await`** es la forma moderna de usarlas y **`Promise.all`** lanza varias a la vez.

## 5.2 Actividades propuestas

1. **Detective de JSON.** En `02_json_sintaxis.js` añade tres textos con errores distintos a los del ejemplo y predice el resultado antes de ejecutarlo. Después corrígelos.
2. **Ida y vuelta.** Crea un objeto `reserva` con un cliente, un número de noches, un array de servicios y una propiedad `notasInternas`. Conviértelo a JSON sin `notasInternas` y reconstruye el objeto con `JSON.parse`.
3. **Copias.** Con el objeto `ana` de `04_copiar_objetos.js`, haz una copia superficial y otra profunda. Modifica las notas en ambas y explica qué ocurre con el original en cada caso.
4. **Validador.** Escribe una función `esReservaValida(valor)` y pruébala con cinco textos JSON (tres válidos y dos no), usando `try/catch` para el texto roto.
5. **Adivina el orden.** Escribe un programa con tres `console.log` y dos `setTimeout` (uno de 0 ms y otro de 500 ms). Predice el orden de la salida y compruébalo.
6. **De then a await.** Reescribe la cadena de `11_encadenar_then.js` con una función `async` y `try/catch`, y compara las dos versiones.
7. **Más rápido.** Con `buscarLibro`, pide los libros 1, 2 y 3: primero con tres `await` seguidos y después con `Promise.all`. Mide el tiempo de cada versión.

## 5.3 Para saber más

- MDN Web Docs (en español): *JSON*, *try...catch*, *Promesas* y *async function*. `https://developer.mozilla.org/es/`
- El tutorial moderno de JavaScript (es.javascript.info): secciones de *Promesas* y *async/await*. `https://es.javascript.info/`
- `https://json.org` (diagramas de la sintaxis de JSON).

## 5.4 Siguiente paso

Ya conoces el formato (JSON) y las herramientas de asincronía (promesas y `async/await`). En **UT_05.2** las usaremos con la API **Fetch** para pedir y modificar datos de un servidor real. Allí estudiaremos también los fundamentos de HTTP (URL, métodos, códigos de estado y cabeceras) y CORS, que tendrán más sentido una vez que hayas visto `fetch` en funcionamiento.